CipherVeil: Data Protection Using Hybrid AES-RSA Encryption and LSB Steganography

ALLAM RAKESH
Roll No. 23K91A0509
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

BOLLEDDU SHANKER
Roll No. 23K91A0543
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

AGARAM SRI SARVAGNA
Roll No. 23K91A0506
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

B AKSHAY
Roll No. 24K915A0504
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

RAJALAKSHMI
[DESIGNATION] (project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY], India
[EMAIL]

***Abstract*---**Encryption protects a message but shows that a secret exists; hiding alone protects nothing once the carrier is examined. This paper presents CipherVeil, a working web tool that applies both layers: a file is encrypted with AES-256-GCM under a fresh key, the key is wrapped with the receiver's RSA-OAEP public key, and the authenticated bundle is hidden in the least significant bits of a PNG image. On six cover images and four payloads, all 22 runs that fit were restored byte for byte, PSNR ranged from 52.5 to 74.6 dB (51.1 dB at full capacity, within 0.053 dB of a closed-form model), SSIM stayed at or above 0.9929, capacity was 3.0 bits per pixel, and all 240 single-bit tampering trials were rejected. The same tests show the limits: only lossless handling preserves the payload, JPEG, noise, resizing and rotation destroy it, and a chi-square test flagged every embedded image. The scheme is invisible to the eye but not to statistics. The paper reports the design, the measurements and these limits, and compares them with the base paper, which addresses a different problem.

***Index Terms*---**AES-GCM, Cyber security, Hybrid encryption, Image steganography, LSB, PSNR, RSA-OAEP.

I. INTRODUCTION

Sensitive files travel over networks that the sender does not control. Encryption keeps the content unreadable but marks the transfer as worth attacking; steganography hides the existence of the transfer but gives no protection once the carrier is suspected [2], [3]. Symmetric ciphers are fast, yet sharing their key is the weak point, and plain least-significant-bit (LSB) substitution can degrade the cover image and be found by statistical tests [4], [5]. In practice a team encrypts a file with one tool and hopes the attachment is not noticed, or hides unencrypted data and relies on secrecy of the method; neither gives two independent layers.

A typical use is a small organisation that must send an account list or a signed memo to a partner over e-mail or a shared drive that it does not trust. The receiver publishes a public key; the sender picks the file, the key and an ordinary-looking picture, and sends back a PNG that looks like any other picture. Only the holder of the private key can open it, and any modification of the picture in transit is detected rather than silently accepted.

The base paper of this work, ProSt [1], advances steganography and encryption in a provably secure form for an untrusted cloud that computes on hidden bits, but reports no imperceptibility or capacity figures. Work on combined schemes [6], [7] is mostly symmetric-only or hardware oriented. The gap is a practical hybrid AES-RSA plus LSB pipeline, evaluated end to end on capacity, imperceptibility, speed, robustness and detectability, with a tool a person can run. CipherVeil fills that gap. Its contributions are:

1) an authenticated bundle format (CVL1) that carries the file name, size and SHA-256 inside the AES-GCM layer, with the magic bytes as associated data; 2) a hybrid pipeline with a fresh AES-256-GCM key and nonce per message wrapped by RSA-OAEP-SHA256, byte-exact in 22 of 22 runs and tamper-evident in 240 of 240 bit flips; 3) a capacity-aware 1-bit LSB embedder at 3.0 bits per pixel whose closed-form PSNR model matches measurement within 0.053 dB, with PSNR never below 51.1 dB at any fill; 4) an honest evaluation in which only lossless handling survives 18 attacks and a chi-square test flags 30 of 30 embedded images; 5) a full-stack tool (FastAPI and React) with key management, analysis and robustness views and a secret-free log, covered by 39 automated tests and 21 smoke checks. The novelty is integration and evaluation, not new primitives.

Section II surveys related work, Section III describes the method, Section IV reports measurements, and Section V concludes.

A. System Hardware

All numbers were measured on one PC: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11 Pro, CPU only. Other jobs shared the machine, so absolute times vary by roughly 10-30 % between runs.

B. Software Requirements

Python 3.11.9 with FastAPI 0.115.6, SQLAlchemy 2.0.36 (SQLite), cryptography 44.0.0, Pillow 11.0.0, NumPy 2.1.3, scikit-image 0.24.0 and matplotlib 3.9.3; React 18.3.1, Vite 6.0.3 and Tailwind CSS on Node 24.19.0; pytest for tests. No dataset, GPU or external service is needed.

II. LITERATURE SURVEY

Base paper. ProSt [1] (Ahmad and Rass, IEEE Access, 2026) introduces provably secure homomorphic steganography. A client hides secret bits in the LSBs of cover

images at a secret position among random decoy bits; an honest-but-curious cloud then evaluates a Boolean circuit made only of Fredkin gates on every pixel without learning which pixels carry secrets. Theorem 1 gives an adversary advantage of order 1/n in the number n of positions, so n of about 10^6 or more is recommended. The reported experiment is a circuit of 17 gates and 61 wires on a 128x128x3 image, evaluated in 8.38 s on an i5-1135G7 laptop. The paper reports no PSNR, SSIM or capacity, and the receiver is a computing server, not a person who must recover a file.

The security model of ProSt differs from the one used here. ProSt assumes that the cloud knows the data is present and protects its location and the circuit; traditional steganography, and this work, assume the opposite and try to hide the existence of the message. Consequently the metrics also differ: ProSt studies the adversary's advantage, while a steganographic transfer is judged by imperceptibility, capacity and detectability.

Embedding. Simple LSB substitution [8] is the standard baseline; LSB matching [9] changes values by plus or minus one and lowers the number of changes, and syndrome-trellis codes [10] minimise a distortion cost. Textbooks [2] and surveys [3] cover the field. CipherVeil keeps plain sequential substitution because it is transparent and measurable.

Combined schemes. SCReedSolo [6] shuffles the message, encrypts it with Fernet, adds Reed-Solomon codes and embeds it at 3 bits per pixel; it is symmetric-only. Fatemieh et al. [7] implement the lightweight stream ciphers Trivium and Grain-128a in memristive IMPLY logic and demonstrate them in LSB steganography. The batch survey also lists four 2024-2025 works on triple-layer visual cryptography, image-object hiding, chaotic encryption with a CNN, and AES with DWT-DCT [11], [12], [13], [14]; only their titles are available here, so no result is quoted.

Steganalysis. The chi-square attack [4] detects equalised value pairs, RS analysis [5] and weighted-stego estimation [15] estimate the embedded length of LSB replacement. They set the bar for the detectability test in Section IV.

Standards and measures. AES [16] in Galois/Counter Mode [17] gives authenticated encryption; RSA-OAEP follows PKCS #1 [18]. Image quality is reported by SSIM [19] and PSNR [20].

TABLE I
SELECTED RELATED WORK

Work	Approach	Data	Reported result	Limitation
ProSt [1]	LSB bits among decoys, Fredkin gates	128x128x3, 17 gates	8.38 s; advantage order 1/n; no PSNR	cloud compute, not file transfer
SCReedSolo [6]	shuffle, Fernet, Reed-Solomon, LSB	text, audio, image; 512x512 to 21188x30328	3 bpp; PSNR 51.8-57.7 dB	symmetric key only
Memristive [7]	Trivium, Grain-128a in IMPLY logic	five 256x256 grayscale	energy down by up to 42 % and 44 %	hardware design
Simple LSB [8]	substitution	test images	baseline	fragile, detectable
LSB matching [9]	plus or minus 1 changes	test images	fewer changes per bit	not tested here
Chi-square, RS, WS [4], [5], [15]	steganalysis of LSB	natural images	estimate embedded length	attacks, not designs
Survey works [11], [12], [13], [14]	visual crypto, chaos + CNN, AES + DWT-DCT	n/a	titles only	not verified

Positioning. Read together, the works split into three groups: embedding designs that lower the visible or statistical footprint of the change [8], [9], [10], combined schemes that add a cipher or an error-correcting code in front of the embedder [6], [7], and a provably secure construction for a different use [1]. None of them pairs a public-key key transport with an LSB carrier and then measures what an attacker, a lossy channel and a bit flip do to it. That measurement, rather than a new embedder, is the aim of this work.

Research gap. The surveyed work advances steganography and encryption mostly separately, including provably secure constructions. What is missing is a practical hybrid AES-RSA plus LSB pipeline, with a public-key step so that sender and receiver share no secret beforehand, measured end to end on capacity, imperceptibility, speed, robustness and detectability, and offered as a tool. CipherVeil addresses that and does not claim provable security.

III. METHODOLOGY

A. Architecture and Main Flow

CipherVeil is a three-tier web application (Fig. 1). A React console on port 5315 calls a FastAPI service on port 8315 with 24 endpoints; the services crypto, lsb, metrics, pipeline, plots, robustness and benchmark do the work; SQLite holds three tables (users, keypairs, oplog) and the covers, payloads and PNG outputs are files. Passwords are hashed with bcrypt, sessions use a 12-hour JWT, every key and operation is checked for ownership, uploads are capped at 25 MB, and the log never receives keys or file contents.

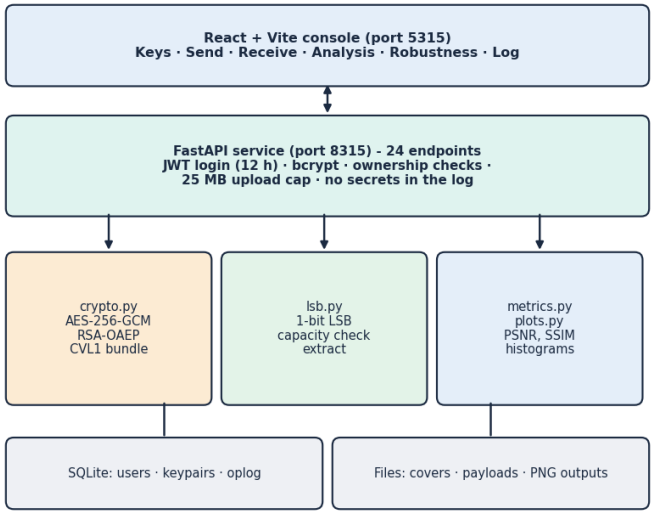


Fig. 1. Architecture of CipherVeil: console, API, services, storage.

The main flow (Fig. 2) has a send path and a receive path. The receiver first generates an RSA keypair (2048, 3072 or 4096 bits) and gives the public PEM to the sender.

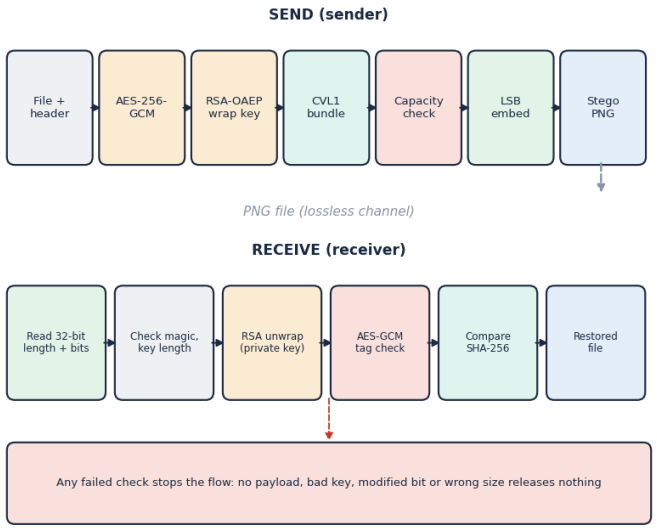


Fig. 2. Main flow: encrypt, wrap the key, check capacity, embed; extract, unwrap, verify.

B. Hybrid Encryption and Bundle

The sender builds a header with the file name, size and SHA-256. A fresh 256-bit AES key and a fresh 96-bit nonce encrypt header and file with AES-256-GCM using the magic bytes CVL1 as associated data. The AES key is wrapped with the receiver's public key by RSA-OAEP with SHA-256 and MGF1-SHA-256. Keys below 2048 bits are refused. The bundle is CVL1 | wrapped-key length (2 B) | wrapped key | nonce (12 B) | ciphertext and tag. For a short file name its overhead is 401, 529 and 657 bytes for RSA-2048, 3072 and 4096. Because the key and nonce are new for every message, the compromise of one message key exposes no other message.

Three choices follow from the problem. GCM is used instead of a plain block-cipher mode because the receiver must be able to tell a modified bundle from a good one: an LSB channel is fragile, and a silent bit error would otherwise yield wrong data. OAEP is used instead of textbook RSA because the wrapped value is a short random key and padding must be randomised and checked. A new key per message is cheap, since the public-key step costs well under a millisecond on the sender's side (Section IV-B), and it removes any need to share a secret before the first message.

C. LSB Embedding and Capacity

The image is read as RGB (any alpha channel is dropped). The bundle length (32 bits) and the bundle are written, most significant bit first, into the lowest bit of each R, G and B value, row by row from the first pixel; the result is saved as PNG. For a W x H image the largest bundle is

$$C = (3WH - 32) / 8 \quad bytes, \qquad (1)$$

that is 3.0 bits per pixel. A larger bundle is rejected before embedding. Extraction reads the length, checks that it is plausible, and returns the bits.

D. Imperceptibility Model

With MSE the mean squared difference over all 3WH values, PSNR is

$$PSNR = 10 \log_{10} (255^2 / MSE). \qquad (2)$$

SSIM [19] is computed per colour channel and averaged. Encrypted bits are close to uniformly random, so each embedded bit flips its value with probability 1/2 and changes it by exactly one. For n embedded bits the expected MSE is n / (6WH), and

$$PSNR = 10 \log_{10} (255^2 \times 6WH / n). \qquad (3)$$

At full capacity n = 3WH and the floor is 51.14 dB. The model depends on the number of bits only, not on the cover, which is what Section IV tests.

E. Extraction and Verification

The receiver reads the length and the bits, checks the magic and the wrapped-key length, unwraps the AES key with the private key, decrypts with AES-GCM and compares the SHA-256 in the header with the hash of the restored file. A wrong key, a flipped bit, a truncated bundle or an image without payload fails a check and releases nothing.

F. Evaluation Protocol

Core evaluation: six covers (200x200 to 1280x720), payloads of 1, 4, 16, 64 and 128 KiB, RSA-3072, three repeats, medians. Fill study: 7 fill levels x 6 covers x 5 random payloads, RSA-2048. Robustness: nine attacks in the application and nine more (lossless re-encodings, WebP, grayscale, rotation, flip, palette) on all six covers, each ending in a real decryption and a hash comparison. Detectability: the chi-square test [4] on the known embedded prefix and the weighted-stego estimate [15] over the whole image, at 10 to 100 % fill. Tamper study: 240 single-bit flips of the stego LSBs, 40 per field of the bundle. Freshness: 50 encryptions of one file. Tests: 39 pytest tests and 21 smoke checks against the running backend.

G. Threat Model and Security Argument

The attacker may read and copy the PNG, run statistical tests on it, modify it, and try keys, but cannot read the receiver's private key. Confidentiality rests on AES-256 and on the RSA-OAEP wrapping of the session key; integrity and authenticity of the content rest on the GCM tag, which covers the header, the file and the magic bytes; the SHA-256 in the header gives an end-to-end check of the restored file. The hiding layer adds a second barrier: an observer must first suspect the picture and then locate and decode the bundle. That barrier is weak by design, because the embedding order is fixed and public, so security does not depend on it. Out of scope are sender authentication (anyone with the public key can send), metadata analysis, compromise of the machine that holds the database, and a quantum adversary against RSA.

H. Implementation and Tests

The services are about 470 lines of Python in seven modules; the five route modules add about 400 lines. The 39 pytest tests cover the cipher (13), the embedder (9), metrics, pipeline and attacks (7) and the API (10); they include exact-fit, oversize, empty-file, non-image, alpha, wrong-key, wrong-size, truncated-bundle and path-traversal cases. The 21-check smoke script drives the running backend through

key generation, send, analysis, receive, wrong key, robustness and the log.

IV. RESULTS

A. Imperceptibility and Capacity

Of 30 cover-payload pairs, 22 fit and 8 were blocked by the capacity check; all 22 were restored byte for byte. PSNR was between 52.5 and 74.6 dB (mean 61.7) and SSIM between 0.9958 and 0.9999 (mean 0.9990). The fill study (Fig. 3a) gives mean PSNR of 68.9, 60.8, 54.1 and 51.15 dB at 1, 10, 50 and 100 % of capacity. The measured values stay within 0.053 dB of the closed form of Section III-D, the standard deviation over five payloads is at most 0.10 dB, and SSIM stays at or above 0.9929, the lowest value occurring on the 1280x720 cover at full capacity. PSNR is therefore above 50 dB at every fill level. Capacity is 3.0 bits per pixel, from 14.6 KiB (200x200) to 337.5 KiB (1280x720). Compared with plain files, encrypted bundles keep PSNR unchanged (about 50 % of embedded bits flip a value either way) but are balanced: the fraction of one-bits is 0.49-0.51 against 0.42-0.46.

Per cover, a 16 KiB payload gives 58.8, 55.9, 58.4, 64.2, 59.0 dB on the astronaut 512x512, cat 451x300, coffee 600x400, generated 1280x720, rocket 640x427 covers (17, 33, 19, 5, 17 % used): the lower the share of capacity used, the higher the PSNR, whatever the picture. The bundle is slightly larger than the file: a 21517 byte memo becomes 21935 bytes (1.9 % more with RSA-2048).

For the 21 KiB memo hidden in the 512x512 photograph of Fig. 4, the analysis screen reports a largest change of 1 of 255 per colour value and 88,010 of 786,432 values changed (11.1911 %), which is about half of the 22.445 % of the capacity used, as the model predicts; the bit-plane view shows that only the lowest plane differs from the cover.

TABLE II
MEASURED RESULTS (SIX COVERS, FOUR PAYLOADS)

Measure	Result
Byte-exact round trips	22 of 22 (8 blocked: too large)
PSNR, core runs (min / mean)	52.5 / 61.7 dB
PSNR at full capacity	51.14-51.17 dB (floor 51.14)
SSIM, core runs (min / mean)	0.9958 / 0.9990
SSIM, lowest at full capacity	0.9929
Capacity	3.0 bpp; 14.6-337.5 KiB
Encrypt / embed / extract (median)	0.15 / 0.25 / 0.05 ms
RSA unwrap + decrypt, 2048 / 3072 / 4096	31 / 94 / 221 ms
Key generation, 2048 / 3072 / 4096	32 / 191 / 354 ms
PNG re-save, WebP lossless, bottom crop 10 %	recovered 6 / 6 each
JPEG q50-q100, noise, resize, left crop, rotate, flip	recovered 0 / 6 each
Chi-square, stego prefix p-value	0.97-1.00 (30 of 30)
Tampered bundles rejected	240 of 240
API send / receive, 512x512, RSA-3072	0.44 s / 0.20 s

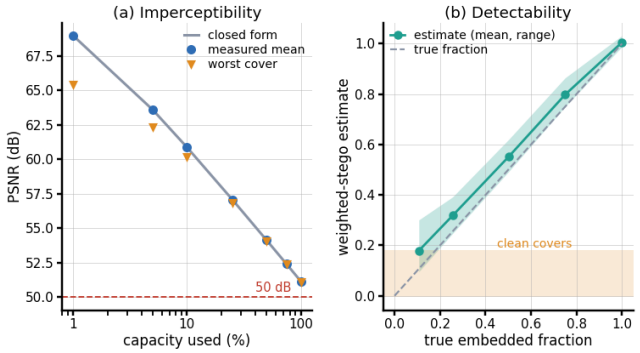


Fig. 3. (a) PSNR against capacity used: mean over six covers, worst cover and closed form. (b) Weighted-stego estimate of the embedded fraction against the true fraction.

B. Speed

Cryptography is cheap: median encryption takes 0.15 ms, embedding 0.25 ms and extraction 0.05 ms. RSA private-key operations dominate: unwrap plus decrypt takes 31, 94 and 221 ms for 2048, 3072 and 4096 bits, and key generation takes 32, 191 and 354 ms (median; the 4096-bit range was 97-541 ms). An API send takes 438 ms on a 512x512 cover and 1.9 s on 1280x720, a receive 198 ms, because SSIM, PSNR and PNG encoding, not the cipher, take most of the time. Other calls are cheap: login 287 ms (bcrypt), key generation through the API 43, 223 and 505 ms, the nine robustness attacks 543 ms, the first render of the histogram and bit-plane plots 942 ms, and the full benchmark over six covers 15 s.

C. Robustness and Failure Cases

Only lossless handling keeps the payload. Of the nine application attacks, a lossless PNG re-save and a bottom crop of 10 % are recovered on all six covers (the crop works only while the bundle lies in the rows that remain); JPEG at quality 95, 75 and 50, Gaussian noise of sigma 1 and 3, resizing to 90 % and a left crop each recover 0 of 6, with LSB error rates of 0.49-0.50, that is, no better than chance. The extra attacks agree: PNG compression level 9, an added alpha channel and WebP lossless recover 6 of 6; JPEG quality 100, lossy WebP, grayscale conversion, rotation, flip and a 256-colour palette recover 0 of 6. The reasons are mechanical: JPEG quantises every 8x8 block and rewrites the lowest bits even at quality 100, noise of one grey level flips about half of the lowest bits, and resizing, rotation, flipping and cropping from the left move every later bit to a wrong position. Images must therefore travel as PNG files, not through messengers or sites that re-encode.

D. Detectability

The chi-square test on the known embedded prefix gave a stego p-value of at least 0.97 in all 30 cases (cover prefix below 0.5 in 27); in three cases the clean cover already looked equalised, so the test did not separate the two. The blind weighted-stego estimate (Fig. 3b) follows the true embedded fraction, for example 0.55 for a true 0.50, but clean covers already read 0.08 to 0.18 on four of six images, so low fill levels are noisy (0.10-0.30 at 10 %). Sequential,

key-independent embedding is detectable at medium and high fill; low fill is safer, not safe.

E. Tamper Resistance

All 240 single-bit flips (40 in each of the length, magic, key-length, wrapped-key, nonce and ciphertext fields) were rejected, none produced wrong data: flips in the wrapped key failed RSA unwrapping, in the nonce or ciphertext failed the GCM tag, and in the length header 13 of 40 were stopped as an invalid header and 27 by the tag. Fifty encryptions of one file gave 50 different nonces and wrapped keys, with 0.50 of the bits differing between bundles.

F. Comparison with the Base and Supporting Papers

The comparison is indicative only, because goals, data and conditions differ. ProSt [1] reports no PSNR, SSIM or capacity, so there is nothing to compare these against; its 8.38 s is the time of a 17-gate homomorphic circuit on 128x128x3, whereas CipherVeil transfers a whole file in milliseconds to a second. CipherVeil takes ProSt's LSB carrier and key-exchange framing but does not implement homomorphic computation, decoy-bit location hiding or a security proof. SCReedSolo [6] reports PSNR of 51.8, 57.7 and 52.9 dB for a text, an audio and an image payload on very different covers at 3 bits per pixel; CipherVeil's 52.5-74.6 dB up to 73 % fill and 51.1 dB at full capacity are of the same order. SCReedSolo adds shuffling and error correction, which CipherVeil lacks; CipherVeil adds public-key key transport and authentication, which SCReedSolo lacks.

G. Output Screens

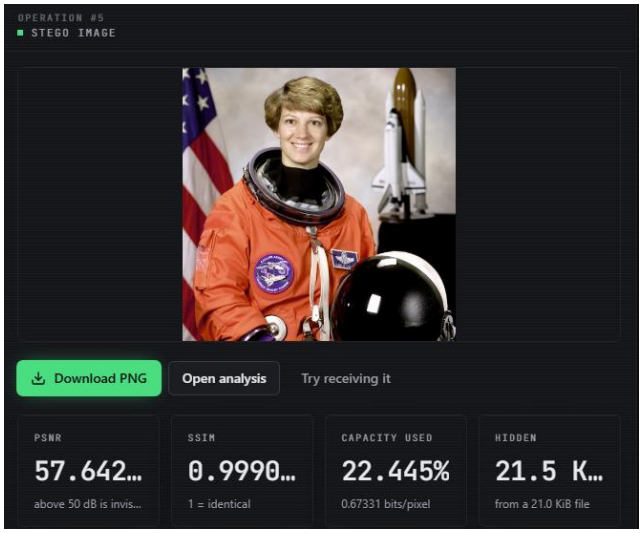


Fig. 4. Send screen: stego image, PSNR 57.642 dB, SSIM 0.999054, 22.445 % capacity used, SHA-256 of the file and stage timings.

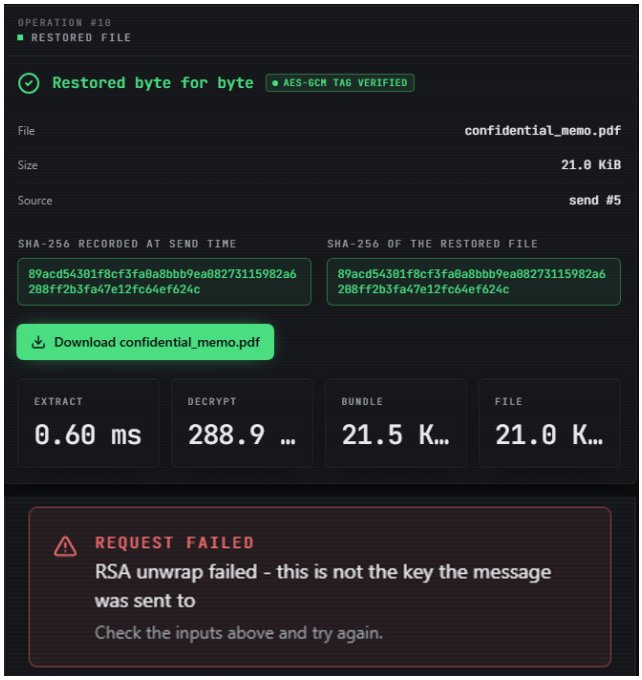


Fig. 5. Receive screen: file restored byte for byte, hashes equal (top); wrong key rejected at RSA unwrap (bottom).

Fig. 4 shows a 21 KiB memo hidden in a 512x512 photograph; Fig. 5 shows the restored file with equal SHA-256 values, and the refusal when the wrong keypair is chosen.

H. Practical Guidance from the Measurements

Three rules follow. First, choose a cover whose capacity is several times the bundle: at about 10 % of capacity PSNR is near 61 dB, at 50 % it is near 54 dB, and the weighted-stego estimate rises with the fill. Second, send the file as a PNG attachment and avoid any service that re-encodes pictures; a single JPEG pass loses the whole payload. Third, prefer RSA-3072: it adds 128 bytes of overhead over RSA-2048 and about 63 ms to the receiver, while RSA-4096 more than doubles the receiver's time again (221 ms). The chi-square result also shows that the equalisation of value pairs, not the visible change, is what gives the embedding away: the mean p-value of the clean prefix falls from 0.26 at 10 % fill to 0.002 at 100 %, while the stego prefix stays at 0.97 or above.

I. Observed Limits

Six committed images and four small synthetic payloads were used, with no camera photographs, one machine and one operating system. Private keys are stored unencrypted in the local database, the JWT secret ships as a placeholder, login has no rate limit, there is no sender signature, and the Send page crashed for an account with no keys because of a missing icon import (found while testing and not fixed).

V. CONCLUSION AND FUTURE SCOPE

CipherVeil meets its six objectives: a hybrid AES-256-GCM and RSA-OAEP pipeline with key generation and exchange, LSB embedding with a capacity check, exact

extraction and decryption, and measured PSNR, SSIM, capacity, speed and robustness. All 22 runs were byte-exact, PSNR never fell below 51.1 dB at any fill, and 240 of 240 tampered bundles were rejected. The key finding is the gap between the two layers: the encryption layer is strong and authenticated, while the hiding layer is invisible to the eye but detectable by simple statistics and destroyed by any lossy step. The limits are those of Section IV-I: plain sequential embedding, six covers, unencrypted key storage and no sender authentication.

Against the stated objectives, the pipeline, the key mechanism, the embedder, the extraction path and the PSNR and SSIM measurements are met in full; the capacity, time and robustness objective is met as a measurement, and the measurement is unfavourable for robustness and detectability. The work does not claim provable security, resistance to modern steganalysers or a gain over the base paper, which solves a different problem.

For a user the outcome is a tool that is safe to use when two conditions hold: the picture travels as a lossless file, and the adversary is not running statistical steganalysis on it. If either condition fails, the AES-GCM and RSA-OAEP layer still protects the content, but the claim of hiding does not hold; the tool says so in its robustness view instead of hiding the fact.

Future work: a key-dependent pixel permutation and LSB matching or adaptive embedding to lower detectability, Reed-Solomon coding for mild channel noise, signed messages and an encrypted key store, and testing against stronger steganalysers on camera photographs.

REFERENCES

[1] S. Ahmad and S. Rass, "ProSt: Provably secure homomorphic steganography," *IEEE Access*, vol. 14, pp. 14393-14409, 2026, doi: 10.1109/ACCESS.2026.3656995.

[2] J. Fridrich, *Steganography in Digital Media: Principles, Algorithms, and Applications*. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[3] N. Provos and P. Honeyman, "Hide and seek: An introduction to steganography," *IEEE Security & Privacy*, vol. 1, no. 3, pp. 32-44, May/Jun. 2003.

[4] A. Westfeld and A. Pfitzmann, "Attacks on steganographic systems," in *Information Hiding (IH 1999)*, Lecture Notes in Computer Science, vol. 1768. Berlin, Germany: Springer, 2000, pp. 61-76.

[5] J. Fridrich, M. Goljan, and R. Du, "Detecting LSB steganography in color and gray-scale images," *IEEE MultiMedia*, vol. 8, no. 4, pp. 22-28, Oct.-Dec. 2001.

[6] S. R. Raiyan and M. H. Kabir, "SCReedSolo: A secure and robust LSB image steganography framework with randomized symmetric encryption and Reed-Solomon coding," arXiv:2503.12368, 2025 (ACPR 2025).

[7] S. E. Fatemieh, R. Shahdi Alizadeh, and E. Zarezadeh, "Design of memristive lightweight encryption for in-memory image steganography," arXiv:2605.03494, May 2026.

[8] C.-K. Chan and L. M. Cheng, "Hiding data in images by simple LSB substitution," *Pattern Recognition*, vol. 37, no. 3, pp. 469-474, Mar. 2004.

[9] J. Mielikainen, "LSB matching revisited," *IEEE Signal Processing Letters*, vol. 13, no. 5, pp. 285-287, May 2006.

[10] T. Filler, J. Judas, and J. Fridrich, "Minimizing additive distortion in steganography using syndrome-trellis codes," *IEEE Trans. Information Forensics and Security*, vol. 6, no. 3, pp. 920-935, Sep. 2011.

[11] B. Kukreja and S. Malik, "An steganography-based triple layered image data hiding using visual cryptography," 2024 (authors as listed on the Authorea preprint page; venue and year of formal publication not confirmed).

[12] "Steganography - data hiding using image object," *Gurukul International Multidisciplinary Research Journal*, 2024.

[13] "Hybrid steganography: Leveraging chaotic encryption and CNN for robust image hiding," in *Proc. 7th Int. Conf. Mechatronics, Robotics and Automation (ICMRA)*, 2024.

[14] "Enhanced image steganography with AES encryption and hybrid DWT-DCT transform for secure data hiding," *International Journal of Advanced Research*, 2025.

[15] J. Fridrich and M. Goljan, "On estimation of secret message length in LSB steganography in spatial domain," in *Proc. SPIE*, vol. 5306, Security, Steganography, and Watermarking of Multimedia Contents VI, 2004, pp. 23-34.

[16] Advanced Encryption Standard (AES), Federal Information Processing Standards Publication 197, NIST, Nov. 2001.

[17] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[18] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA cryptography specifications version 2.2," IETF RFC 8017, Nov. 2016.

[19] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Trans. Image Processing*, vol. 13, no. 4, pp. 600-612, Apr. 2004.

[20] Q. Huynh-Thu and M. Ghanbari, "Scope of validity of PSNR in image/video quality assessment," *Electronics Letters*, vol. 44, no. 13, pp. 800-801, Jun. 2008.